

Secure Messaging Platform

(Signal Clone) — SDE Fullstack Assignment

Description

Build a functional clone of the Signal messaging application that replicates Signal's design, user experience, and core messaging workflows.

The platform should allow users to register, manage contacts, create one-on-one and group conversations, and send and receive messages in real time, all within the clean, privacy-focused interface of the original Signal app.

Your implementation should visually and functionally feel like a modern Signal Messenger. The focus is on recreating the Signal user experience and core messaging workflows rather than implementing real end-to-end cryptographic protocols — encryption can be mocked or simulated.

Note: Your application should match the original application in terms of UI and UX, look and feel should be exactly the same.

AI Tools Usage

You are allowed and encouraged to use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or any other AI assistant for development. Use AI as heavily as you like to move fast. However, you must understand every line of code you submit and be prepared to explain your implementation decisions during the evaluation interview.

Technical Stack

- Frontend: Next.js (TypeScript)
- Backend: Python with FastAPI / Django
- Database: SQLite (design your own schema)
- Real-time: WebSockets (or any real-time mechanism of your choice)

Core Features (Must Have)

1. Authentication / Onboarding

Implement a simple mocked authentication / registration flow.

- Register with a phone number or username (verification can be mocked with a fixed OTP)
- Set a display name and profile avatar
- Login / Logout and session persistence

Real phone verification and actual cryptographic key exchange can be mocked.

2. Contacts & Conversation List

Recreate the Signal main view with a left-hand conversation/chat list.

- View list of conversations sorted by most recent activity

- Search conversations and contacts
- Add a new contact
- Unread indicators and last-message preview
- Online / last-seen indicators (can be mocked)

3. One-on-One Messaging

Implement real-time direct messaging between two users.

- Send and receive text messages in real time
- Message timestamps
- Delivery / read receipts (the single/double check experience)
- Typing indicators
- Message status: sending, sent, delivered, read
- All messages must persist in the database

4. Group Messaging

- Create a group with a name and members
- Send and receive messages in a group
- View group members
- Add / remove members (admin controls)
- All group data and messages must persist

5. Signal Experience

The application should closely resemble the Signal experience, including:

- Navigation and layout (conversation list + chat pane)
- Message bubbles and threading
- Forms, modals, search, and filters
- Notifications / toasts
- Settings placeholders (privacy, notifications, appearance)

The goal is to make the application feel like Signal rather than a generic chat app.

Mocked / Placeholder Sections

The following can be present as placeholders (a simple “Coming Soon” is sufficient):

- Voice / Video calls
- Stories
- Linked devices
- Actual end-to-end encryption (simulate or mock)

Bonus (Optional)

- Attachments (images / files)
- Message reactions (emoji)
- Reply-to / quoted messages
- Disappearing messages (functional)

- Dark mode
- Responsive design (mobile, tablet, desktop)
- Keyboard shortcuts

Important Notes

- UI Design: your application should totally resemble Signal's design. Study Signal's UI carefully before starting.
- Sample Data: seed your database. Seed multiple users, conversations, and messages so the app is immediately usable.
- Database Design: design your own database schema. This will be evaluated.
- README File: include setup instructions, tech stack used, architecture overview, database schema, and any assumptions made.
- Original Work: plagiarism from existing repositories will result in immediate disqualification.

Deliverables

- Source Code: a public GitHub repository containing frontend/ and backend/.
- Documentation: a README with setup instructions, architecture overview, database schema, and API overview.
- Demo: a hosted, working link.

Submission

- Upload your code to GitHub and ensure the repository is public.
- Deploy your application (Vercel, Netlify, Render, Railway, or any cloud service).
- Submit both the GitHub repository link and the deployed application link.

Evaluation Criteria

Criteria	What We Look For
Functionality	All core features working correctly, including real-time messaging
UI/UX	Visual similarity to the original app's design and UX patterns
Database Design	Well-structured schema with proper relationships
Backend / API Design	Clean, sensible API design and architecture
Code Quality	Clean, readable, and well-organized code
Code Modularity	Proper separation of concerns, reusable components
Code Understanding	Ability to explain your code during evaluation

Timeline

Estimated effort: approximately 24 hours of work.

Submission Deadline: as communicated alongside this assignment.

Good luck!